AI-Powered Legal Document Parser and Risk Analyzer

A Report Submitted in Partial Fulfilment of the Requirements of the Summer Internship Programme

Submitted by

Soureen Majumder

Bachelor of Engineering

Instrumentation and Electronics Engineering (2nd Year)

Jadavpur University, Kolkata

Under the Mentorship of

Dr. Chandan Biswas

Period of Internship: 18th May 2026 – 31st July 2026

Report Submitted to

IDEAS – Institute of Data Engineering, Analytics and Science Foundation

Indian Statistical Institute (ISI), Kolkata

July 30, 2026

Acknowledgement

I would like to express my sincere gratitude to **Dr. Chandan Biswas** for his invaluable mentorship, guidance, and continuous encouragement throughout the course of this internship. His insightful feedback and willingness to share his expertise were instrumental in shaping both the technical direction of this project and my own understanding of applied artificial intelligence and cloud-based system design.

I am equally thankful to **IDEAS – the Institute of Data Engineering, Analytics and Science Foundation**, and the **Indian Statistical Institute (ISI), Kolkata**, for providing me with the opportunity to undertake this internship and for furnishing an intellectually stimulating environment in which to learn, experiment, and grow. The structured training sessions, hands-on exposure to real-world problems, and collaborative culture fostered by the institute have significantly enriched my learning experience.

My sincere thanks also go to the faculty and staff at IDEAS who conducted the foundational training sessions on data science, machine learning, cloud computing, and related domains, which laid the groundwork necessary to undertake this project with confidence.

I would further like to acknowledge **Jadavpur University** and the **Department of Instrumentation and Electronics Engineering** for their continued academic support, and for cultivating the foundational knowledge that enabled me to pursue this internship opportunity.

Finally, I extend my heartfelt appreciation to my family, friends, and peers, whose constant encouragement and support served as a source of motivation throughout this endeavour. This internship has been a rewarding and formative experience, and I am grateful to everyone who contributed to its successful completion.

Soureen Majumder

Contents

Abstract

The **AI Document Parser and Risk Analyzer** is an intelligent software system engineered to automate the analysis of legal and contractual documents, thereby alleviating the burden of exhaustive manual review. The system ingests uploaded documents, extracts their underlying textual content, and subsequently generates concise summaries alongside structured risk assessments through the application of Large Language Models (LLMs). It is capable of identifying latent legal, financial, compliance, and operational risks embedded within dense contractual language. Architecturally, the platform comprises a Streamlit-based user interface, an Amazon Web Services (AWS) API Gateway, an AWS Lambda function for serverless backend processing, and the Claude API for advanced natural language understanding. Textual content is extracted from PDF documents, image files, and scanned PDFs prior to transmission to the backend for analysis, where document summarization and risk identification are performed as two distinct, independently optimized AI processing stages to improve both accuracy and maintainability. The resulting output is rendered in a clean, organized format that allows users to rapidly assimilate key information and potential points of concern. By virtue of its serverless architecture, the system achieves elastic scalability, minimal infrastructure overhead, and cost-efficient deployment. Taken as a whole, this project illustrates the practical convergence of artificial intelligence, cloud computing, and natural language processing in the domain of legal document analysis, substantially reducing manual effort, enhancing the efficiency of document review, and empowering users to arrive at better-informed decisions.

1. Introduction

The exponential growth in the volume of digital documentation across virtually every industry has given rise to a pressing demand for intelligent systems capable of automatically processing and interpreting textual information. Legal contracts, agreements, policy documents, and compliance reports are frequently characterized by dense, technical language and clauses of considerable consequence that necessitate careful scrutiny. The manual analysis of such documents is not only labour-intensive and time-consuming but also inherently vulnerable to human oversight and error. Recent advances in Artificial Intelligence (AI), and Large Language Models (LLMs) in particular, have made it feasible to automate document comprehension, summarization, and contextual risk identification with markedly improved efficiency and precision.

This internship project, the **AI Document Parser and Risk Analyzer**, was conceived and developed to furnish an automated solution for extracting meaningful insight from legal and contractual documents. The application permits users to upload documents, whereupon the system extracts the textual content, produces a concise summary, and identifies potential legal, financial, compliance, and operational risks. Results are presented in a structured and intuitive format that enables users to rapidly comprehend lengthy documents and direct their attention toward areas warranting closer examination.

The project is built upon a modern, serverless cloud architecture. The user interface was developed using **Streamlit**, furnishing an interactive platform for document upload and result visualization. The backend was implemented using **AWS Lambda**, enabling the serverless execution of the document analysis logic. Communication between the frontend and backend is mediated by **Amazon API Gateway**, while the **Claude Large Language Model** performs document summarization and intelligent risk analysis by means of advanced natural language processing techniques. Python served as the primary programming language for the implementation of both the frontend and backend components.

A review of existing document analysis tools reveals that many conventional systems rely upon keyword matching, rule-based extraction, or predefined templates to process documents. While

such approaches are useful for extracting structured information, they frequently fail to grasp contextual meaning and to surface implicit risks embedded within complex legal language. Modern AI-based approaches, particularly those grounded in Large Language Models, offer substantially enhanced capabilities by discerning semantic relationships, producing human-like summaries, and detecting context-dependent risks. This project harnesses these advancements to build an intelligent document analysis system capable of assisting users in reviewing lengthy legal documents with far greater effectiveness.

The development process encompassed the extraction of text from uploaded PDF documents, the transmission of the extracted content to the backend via a REST API, the processing of that text through AI-based summarization and risk analysis modules, and the presentation of the generated output through the Streamlit interface. The modular architecture adopted throughout development enhances maintainability, scalability, and ease of future enhancement, while simultaneously reducing infrastructure management overhead through serverless cloud deployment.

This internship afforded valuable, hands-on exposure to cloud computing, REST API development, serverless architecture, AI model integration, and intelligent document processing. It demonstrated, in concrete terms, how contemporary AI technologies can be integrated with cloud services to construct scalable applications capable of efficiently solving real-world document analysis problems.

1.1 Technologies Employed

The principal technologies employed during the development of the project include:

- Python
- Streamlit
- Amazon Web Services (AWS)
- AWS Lambda
- Amazon API Gateway
- Claude Large Language Model (LLM)
- REST API
- PDF Text Extraction Libraries
- OCR (Tesseract) for scanned documents and images
- JSON
- Cloud Computing

- GitHub for Version Control

1.2 Foundational Training

During the initial two weeks of the internship, structured training sessions were conducted to establish a robust foundation in Data Science, Machine Learning, Artificial Intelligence, Cloud Computing, and Software Development. The topics covered during this training phase are enumerated below:

1. Basic Statistics for Data Science
2. Data Visualization
3. Introduction to GitHub and Cloud Computing
4. Data Science Application Development using Streamlit
5. Machine Learning
    - Supervised Learning
    - Unsupervised Learning
    - Regression
    - Classification
    - Clustering
6. Large Language Model (LLM) Fundamentals
7. Deep Learning
8. Computer Vision
9. Agentic AI
10. Reinforcement Learning
11. System Overview of Data Science
12. Communication Skills

The knowledge acquired during these training sessions furnished the theoretical and practical foundation essential to the successful development of the AI Document Parser and Risk Analyzer, particularly with respect to cloud deployment, application development, AI model integration, and intelligent document processing.

2. Project Objective

2.1 Develop an Intelligent Document Parsing System

- Design a system capable of accepting legal and contractual documents in digital format.
- Extract textual content from uploaded documents for further processing.
- Preprocess the extracted text to render it suitable for AI-based analysis.
- Ensure compatibility with commonly used document formats such as PDF.

2.2 Automate Document Summarization

- Generate concise summaries of lengthy documents using a Large Language Model (LLM).
- Highlight the key clauses and salient information contained within the document.
- Reduce the time required for users to comprehend complex legal documents.
- Improve readability while faithfully preserving the context and meaning of the original document.

2.3 Perform AI-Based Risk Analysis

- Identify potential legal, financial, operational, and compliance-related risks.
- Classify the detected risks into meaningful, well-defined categories.
- Provide explanations for each identified risk so as to improve interpretability.
- Assist users in pinpointing critical sections that require detailed review.

2.4 Develop a Scalable Cloud-Based Application

- Build an interactive frontend using Streamlit for document upload and result visualization.
- Implement backend processing using AWS Lambda.
- Integrate Amazon API Gateway to enable secure communication between the frontend and backend.
- Utilize a serverless architecture to achieve scalability and simplify deployment.

2.5 Improve the Efficiency of Document Review

- Reduce the manual effort involved in reviewing lengthy legal documents.
- Present structured, user-friendly analysis results to support sound decision-making.
- Minimize the likelihood of overlooking important clauses and latent risks.
- Demonstrate the practical application of Artificial Intelligence, Natural Language Processing, and cloud computing to document analysis.

3. Methodology

3.1 Methodology Overview

The development of the **AI Document Parser and Risk Analyzer** followed a systematic, iterative software development methodology that integrated principles of Artificial Intelligence (AI), Natural Language Processing (NLP), cloud computing, and modern web application development. The methodology was designed to ensure that the final application could efficiently process legal documents, generate concise summaries, identify potential risks, and present the resulting insights through an intuitive web interface.

The project commenced with an extensive study of existing approaches to document parsing and legal document analysis. Traditional document review systems rely predominantly on manual examination or rule-based text extraction techniques, both of which demand considerable human effort and are frequently incapable of grasping the contextual meaning of legal language. These limitations underscored the need for an intelligent system capable of performing genuine semantic analysis rather than superficial keyword matching.

Having clarified the problem statement, the overall system architecture was planned by decomposing the application into independent functional modules. This modular design philosophy was adopted because it simplifies maintenance, enhances scalability, and permits each component of the system to be developed and tested in isolation. The principal modules identified during the design phase were as follows:

- User Interface Module
- Document Processing Module
- Text Extraction Module
- API Communication Module
- AI Analysis Module
- Risk Analysis Module
- Result Visualization Module

- Cloud Deployment Module

Partitioning the project into these modules ensured that alterations made to any single component would exert minimal impact on the remaining components. For instance, modifications to the user interface could be implemented without disturbing the backend analysis logic, while refinements to the AI prompts could be incorporated without necessitating a redesign of the frontend application.

The development process adhered to an incremental methodology. Rather than implementing all features simultaneously, the project was constructed in successive stages. A basic Streamlit application was first created to provide users with a document upload interface. Once the frontend had been established, functionality for extracting textual information from uploaded PDF files was integrated using dedicated Python libraries. After reliable text extraction had been achieved, communication between the frontend and backend was established through a REST API.

The backend processing module was developed using AWS Lambda, thereby enabling the serverless execution of the application logic. The Lambda function receives the extracted document text from the frontend by way of Amazon API Gateway, processes the incoming request, interacts with the Claude Large Language Model, and returns structured analysis results in JSON format.

The AI analysis stage was deliberately divided into two major processing tasks:

Document Summarization

The extracted document text is analysed by the Claude Large Language Model to generate a concise summary that highlights the important clauses, obligations, responsibilities, and other significant information contained within the document. This summary enables users to comprehend lengthy legal documents without the need to read every page manually.

Risk Identification

Following document summarization, the same document is subjected to further analysis to identify potential legal, operational, financial, and compliance-related risks. Rather than merely searching for predefined keywords, the AI model performs a contextual reading of the document and generates descriptive explanations for each detected risk. This substantially improves the quality of analysis in comparison with conventional rule-based systems.

Upon receiving the AI-generated response, the frontend application parses the returned JSON

data and displays the summary and identified risks in clearly delineated sections. Additional visual indicators, such as risk counts and severity labels, enhance readability and assist users in prioritizing the most critical observations.

Throughout the development process, extensive testing was conducted following the completion of each module. Individual components were first tested in isolation to verify their correct functioning prior to integration into the complete application. This modular testing strategy reduced debugging complexity and improved the overall reliability of the system.

Error-handling mechanisms were incorporated at multiple stages of the application. Invalid document uploads, empty files, API communication failures, backend exceptions, network timeouts, and malformed responses were all accounted for during implementation. User-friendly error messages were displayed whenever unexpected situations arose, ensuring that users received meaningful feedback rather than encountering application crashes.

The project methodology also placed considerable emphasis on scalability and maintainability. By employing AWS Lambda and Amazon API Gateway, the backend infrastructure remains decoupled from the frontend application and can be scaled automatically in response to workload demands. Likewise, the adoption of a REST-based architecture allows future mobile applications or additional frontend platforms to reuse the same backend services without requiring substantial architectural modification.

In summary, the adopted methodology integrates document processing, artificial intelligence, cloud computing, and modern software engineering practices into a unified workflow capable of automating legal document analysis while minimizing manual effort and enhancing the efficiency of document review.

3.2 Requirement Analysis

Requirement analysis constitutes one of the most critical phases of software development, as it establishes the functional and technical foundation upon which the entire project rests. Prior to implementation, the requirements of the AI Document Parser and Risk Analyzer were carefully examined in order to determine the expected functionality, system architecture, software tools, cloud services, and computational resources necessary for successful development.

The primary problem identified during the analysis phase was the inherent difficulty of manually reviewing lengthy legal and contractual documents. Legal agreements frequently span numerous pages and contain intricate clauses that are difficult for non-experts to interpret. Manual examination is not only time-consuming but also increases the likelihood of overlooking im-

portant obligations or hidden risks.

On the basis of this problem statement, several functional requirements were identified.

Functional Requirements

- The system should allow users to upload legal documents in digital format.
- Uploaded PDF documents should be processed automatically, without requiring manual intervention.
- The system should accurately extract textual information from uploaded PDF files.
- The extracted document text should be transmitted securely to the backend server.
- The backend should generate a concise summary of the uploaded document using Artificial Intelligence.
- The backend should identify important legal, financial, compliance, and operational risks present within the document.
- The generated results should be returned in a structured JSON format.
- The frontend should display document summaries and risk analysis results within a clean, user-friendly interface.
- The application should provide appropriate error messages whenever invalid inputs or communication failures occur.

In addition to these functional requirements, several non-functional requirements were also considered during system design.

Non-Functional Requirements

- The application should provide a responsive and intuitive user interface.
- The backend architecture should support scalability through serverless cloud computing.
- Communication between the frontend and backend should be both secure and efficient.
- The application should minimize response time while preserving analysis quality.
- The system should be modular in design, so as to simplify future enhancements.
- The application should remain maintainable through a clear separation of frontend and backend responsibilities.

- The project should utilize publicly available cloud infrastructure for deployment.

The software components selected during development were evaluated on the basis of ease of implementation, documentation availability, cloud compatibility, community support, and integration capability.

A further important consideration during requirement analysis concerned the selection of an appropriate Artificial Intelligence model for document understanding. Several alternatives were initially studied, including the development of a custom Natural Language Processing pipeline and the training of a machine learning model specifically tailored to legal document classification. However, such approaches would necessitate a large annotated dataset, extensive computational resources, and considerable model training time.

Given the constraints of the internship duration and the stated project objectives, integrating a pre-trained Large Language Model emerged as the more practical and efficient solution. Large Language Models possess strong natural language understanding capabilities and are able to perform summarization, contextual interpretation, and reasoning without requiring project-specific model training. Accordingly, the Claude API was selected as the core AI engine responsible for document summarization and risk identification.

The requirement analysis phase also surfaced several technical limitations. The application relies upon textual content extracted from PDF documents; consequently, scanned image-based PDFs cannot be processed reliably using conventional text extraction libraries, as such documents lack machine-readable text. Although Optical Character Recognition (OCR) techniques were investigated during development, practical constraints related to computational overhead and processing time precluded their full integration into the deployed system. The implemented version therefore primarily supports digitally generated PDF documents containing selectable text.

Cloud infrastructure requirements were likewise analysed prior to deployment. Since AI inference is computationally intensive, executing the backend through serverless cloud services conferred several advantages, including automatic scalability, reduced infrastructure maintenance, and simplified deployment. AWS Lambda was accordingly selected for backend execution, while Amazon API Gateway was employed to expose REST endpoints accessible securely by the Streamlit frontend.

The requirement analysis phase ultimately established the functional scope, technology stack, architectural decisions, and implementation strategy that guided the subsequent development of the project.

3.3 Software and Tools Used

The AI Document Parser and Risk Analyzer integrates a diverse array of software frameworks, cloud platforms, programming libraries, and artificial intelligence technologies. Each tool was selected after careful consideration of factors such as compatibility, ease of development, scalability, documentation support, and integration capability.

Python

Python served as the primary programming language for the entire project. Its extensive ecosystem of libraries, simplicity of syntax, and excellent support for artificial intelligence and cloud computing rendered it an ideal choice for both frontend and backend development. Python was employed to implement document processing, API communication, JSON parsing, user interface logic, backend services, and interaction with the Claude API.

Streamlit

The frontend application was developed using Streamlit, an open-source Python framework designed for the rapid development of interactive web applications. Streamlit permitted the creation of a clean user interface without requiring expertise in HTML, CSS, or JavaScript. Users are able to upload documents, initiate analysis, monitor processing progress, and visualize generated summaries and identified risks through an intuitive interface. The framework also provides widgets such as file uploaders, buttons, progress bars, expandable containers, metrics, and tabbed layouts, all of which were utilized during implementation to enhance the overall user experience.

AWS Lambda

AWS Lambda was selected as the backend execution environment on account of its serverless architecture. Rather than maintaining dedicated backend servers, AWS Lambda executes application logic only upon the receipt of incoming requests, substantially reducing infrastructure management while providing automatic scalability in response to workload. The Lambda function performs several critical operations, including receiving document text, generating prompts, communicating with the Claude API, formatting responses, and returning structured JSON data to the frontend.

Amazon API Gateway

Amazon API Gateway functions as the communication bridge between the Streamlit frontend and the AWS Lambda backend. It exposes REST API endpoints that receive HTTP POST requests containing document text; following invocation of the Lambda function, the generated analysis results are returned through the same API endpoint to the frontend application. The use of API Gateway also affords flexibility for future integration with mobile applications or additional frontend platforms.

Claude Large Language Model

The core intelligence of the application is furnished by the Claude Large Language Model. Rather than training a custom Natural Language Processing model from scratch, the project harnesses Claude through its API to perform advanced language understanding tasks. Claude performs two principal operations:

- Document summarization
- Context-aware risk identification

Its capacity to interpret complex legal language enables the application to identify risks on the basis of semantic understanding rather than simple keyword matching.

PyPDF2

PyPDF2 was employed to extract machine-readable text from uploaded PDF documents. The library reads each page individually and retrieves textual information while preserving the original reading order as faithfully as possible. The extracted text constitutes the input subsequently supplied to the AI model for further analysis.

Requests Library

The Requests library was used to establish communication between the Streamlit frontend and the backend REST API. HTTP POST requests containing extracted document text were transmitted to Amazon API Gateway, while JSON responses generated by AWS Lambda were received and processed for visualization.

JSON

JSON served as the primary format for exchanging information between the frontend and backend components. AI-generated responses were organized into structured JSON objects containing document summaries, identified risks, severity information, and additional metadata. JSON was selected on account of its lightweight structure, readability, and widespread support across programming environments.

GitHub

GitHub was utilized as the version control platform throughout development. It enabled the systematic management of source code, preserved a complete version history, facilitated the backup of project files, and simplified the tracking of modifications made during iterative development.

Development Environment

The application was developed using Visual Studio Code as the primary Integrated Development Environment (IDE). Local testing was conducted prior to the deployment of backend services to AWS, and Python virtual environments were used for dependency management, ensuring consistent execution across differing development environments.

Cloud Services

The project makes use of multiple AWS cloud services in order to implement a scalable architecture:

- AWS Lambda for serverless backend execution.
- Amazon API Gateway for REST API management.
- AWS Identity and Access Management (IAM) for permission control.
- Amazon CloudWatch for logging, debugging, and the monitoring of backend execution.

Taken together, these cloud services provide a reliable, scalable, and maintainable deployment environment well suited to AI-powered document analysis applications. The combination of these software tools enabled the development of an integrated, intelligent document analysis system capable of automating legal document summarization and contextual risk identification, while preserving modularity, scalability, and ease of future enhancement.

3.4 System Architecture

The AI Document Parser and Risk Analyzer is structured as a five-layer, loosely coupled system in which each layer communicates with its neighbours through a well-defined interface — a design choice that allows any single layer to be modified, replaced, or scaled without disturbing the rest of the application.

- **Presentation Layer** — A Streamlit application (`app.py`) responsible for document upload, in-app preview, tabbed result visualization, and report downloads. Streamlit was chosen over a traditional HTML/CSS/JavaScript frontend because it allows a data-oriented Python codebase to expose a fully interactive web UI without context-switching between languages, while still permitting deep visual customization through injected CSS — evident in the bespoke navy-and-serif "LexAI" brand theme, custom metric cards, and colour-coded risk badges implemented directly in `app.py`.
- **API Layer** — Amazon API Gateway, which exposes a single REST endpoint (`POST /prod/`) and additionally answers the `OPTIONS` pre-flight request required by the browser's Cross-Origin Resource Sharing (CORS) policy before the actual POST is permitted.
- **Compute Layer** — An AWS Lambda function (`lambda_function.py`) that receives the extracted document text, orchestrates two independent calls to the Claude API, normalizes their output, and returns a single structured JSON payload.
- **AI Layer** — The Claude Large Language Model, accessed through the official `anthropic` Python SDK, which performs document summarization and risk identification as two functionally separate prompts rather than a single monolithic request.
- **Reporting Layer** — A ReportLab-based PDF generator (`risk_pdf.py`) that converts the structured risk list into a branded, shareable PDF document, and a companion PDF-rendering module (`doc_preview.py`) built on PyMuPDF that lets the user view the original uploaded document without leaving the browser.

This layered separation mirrors the modular design philosophy described in Section 4.1: the Streamlit layer knows nothing about how risks are detected, the Lambda layer knows nothing about how results are styled, and the AI layer is entirely agnostic to both the transport mechanism and the presentation format. This decoupling is what permits, for example, the PDF report's colour palette to be changed in `risk_pdf.py` without touching a single line of `lambda_function.py`, or the underlying LLM prompt to be refined without requiring any

change to the frontend.

Figure 3.1 presents the complete component-level architecture diagram of the system, mapping every function described above onto the frontend/backend boundary and showing the exact data structures (session-state schema, JSON request/response shapes) exchanged at each interface.

3.5 Workflow of the Proposed System

The end-to-end workflow of the application follows a linear, user-driven pipeline composed of the following stages:

1. The user uploads one or more PDF or TXT files through the Streamlit file uploader.
2. Each file is immediately fingerprinted using a composite key of its name and byte size, and its text is extracted and cached in `st.session_state` so that re-renders of the page (an inherent characteristic of Streamlit's execution model) do not repeat expensive extraction work.
3. The user may expand an in-app preview for any document, rendered page-by-page as images, prior to committing to a full analysis.
4. Upon clicking **Analyse**, the extracted text is transmitted as a JSON payload to the API Gateway endpoint via an HTTP POST request.
5. API Gateway invokes the Lambda function, which concurrently dispatches the text to Claude for (a) summarization and (b) risk analysis.
6. The Lambda function normalizes and validates both AI responses, guaranteeing a consistent schema regardless of how the underlying model formatted its output, and returns the combined result.
7. The Streamlit frontend parses the response, updates the relevant document's record in session state, and re-renders the results across four dedicated tabs: Preview, Summary, Risk Analysis, and Raw Text.
8. The user may filter risks by severity, switch between multiple analysed documents using a radio selector, and download either a plain-text summary or a fully branded PDF risk report.

This workflow was deliberately kept linear and synchronous from the user's perspective — even though the backend performs two AI calls in parallel — so that the perceived experience remains simple: upload, analyse, review, download.

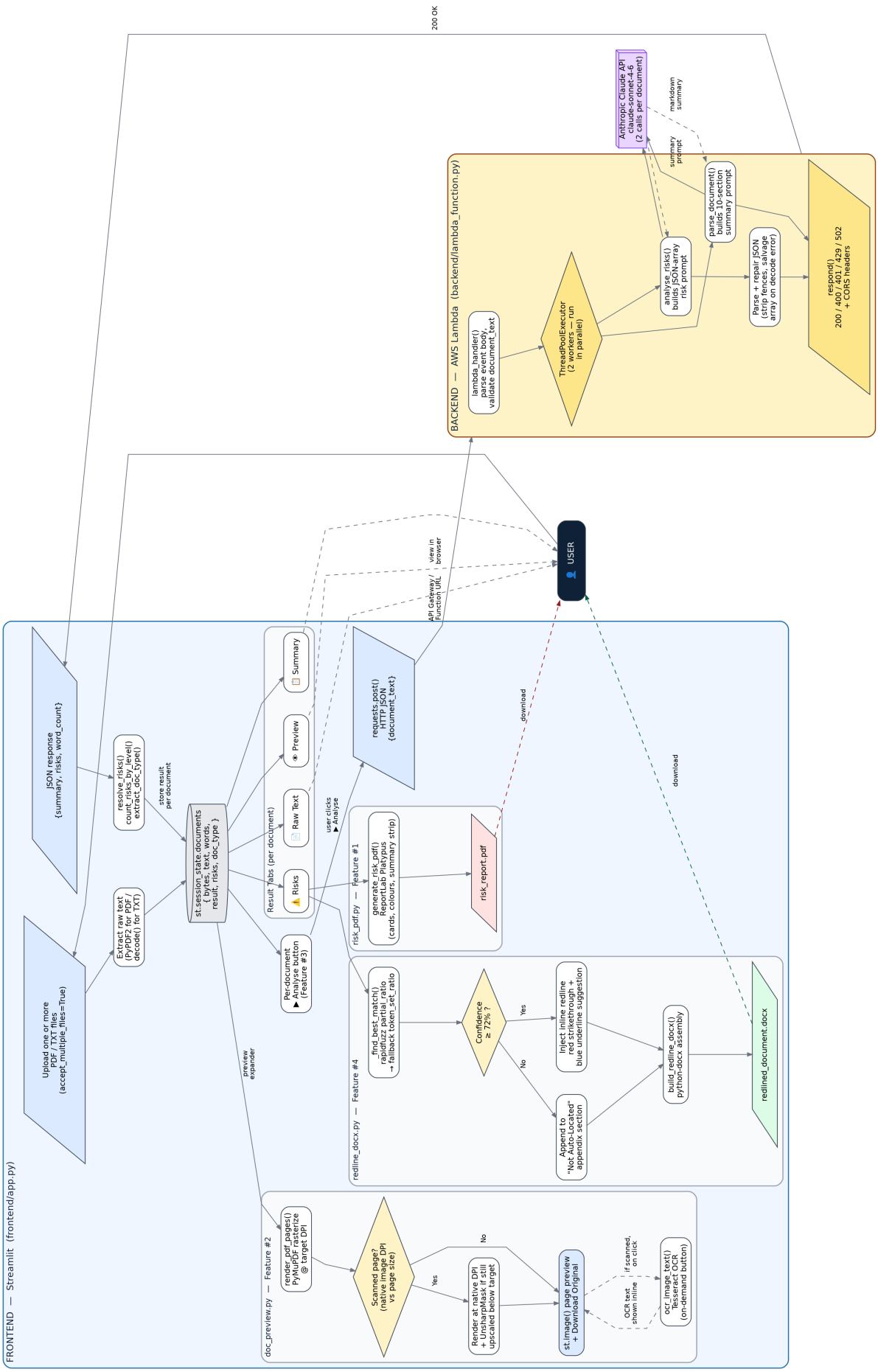


Figure 3.1: Complete system architecture of the AI Document Parser and Risk Analyzer, showing the Streamlit frontend (document upload, session-state management, preview, and result tabs), the AWS Lambda backend (concurrent Claude API calls for summarization and risk analysis), and the supporting modules for PDF preview, risk-report generation, and redlined DOCX export.

3.6 Data Collection

Unlike conventional machine learning projects, this system does not depend on a curated training dataset, since it uses a pre-trained Large Language Model rather than a custom-trained classifier. The "data" relevant to this project therefore consists of the corpus of real-world legal documents used to validate the pipeline during development, including Non-Disclosure Agreements (NDAs), employment contracts, vendor and service agreements, and residential lease agreements. These documents were deliberately chosen to span a range of lengths, clause structures, and drafting styles, so that the summarization and risk-detection prompts could be evaluated against genuinely varied legal language rather than a narrow, homogeneous sample. Both digitally generated PDFs (with embedded, selectable text) and, separately, scanned image-based PDFs were collected during testing in order to characterize the system's current text-extraction boundary, described further in Section 4.18.

3.7 Document Upload and Processing

Figure 3.2 shows the application's landing screen, which orients the user with a three-step "Upload → Analyse → Review" indicator, a sidebar usage guide, and the initial document drop zone.

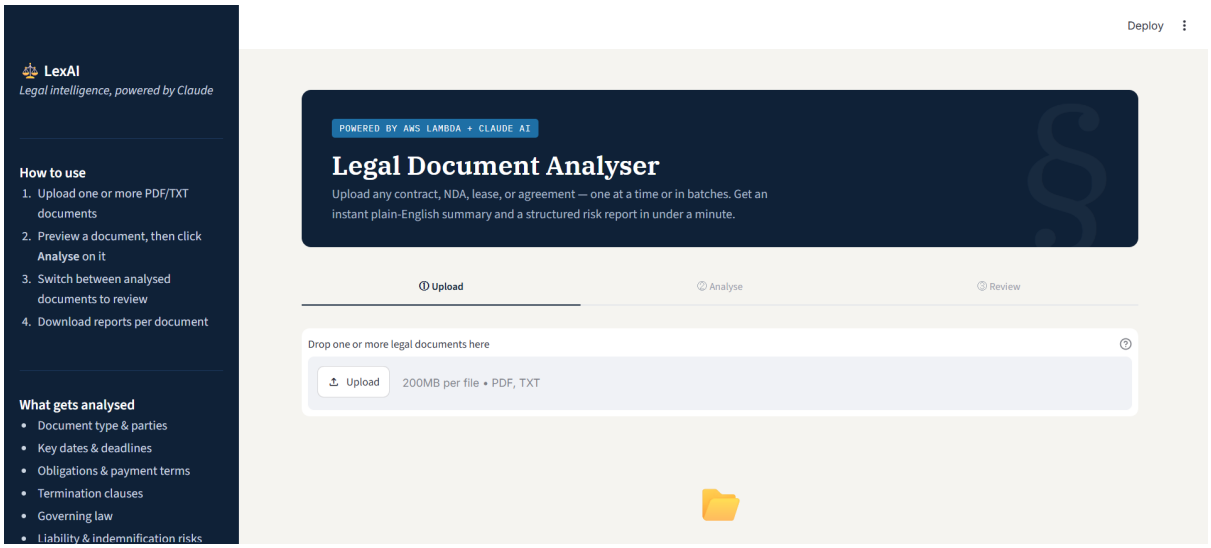


Figure 3.2: LexAI landing screen showing the document upload step, sidebar instructions, and the list of risk categories analysed.

Document ingestion is handled through Streamlit's `st.file_uploader` widget, configured with `accept_multiple_files=True` so that users may submit an entire batch of documents in a single interaction rather than being restricted to one file at a time. Each uploaded

file is keyed by a composite identifier (`filename__filesize`) and stored as a structured record inside `st.session_state.documents`, capturing the raw file bytes, extracted text, word count, file size, file type, and — once analysed — the AI-generated summary, risk list, and severity counts.

This session-state-backed design serves two practical purposes. First, it prevents redundant, costly re-extraction of text every time Streamlit reruns the script (which happens on every user interaction), since previously processed files are recognised by their key and skipped. Second, it keeps the application state synchronized with the file uploader widget itself: if a user removes a file from the uploader, the corresponding record — and any cached OCR or analysis results tied to it — is automatically purged from session state, preventing stale data from lingering in the interface.

Each document is additionally rendered as an interactive overview table showing its name, type, size, word count, AI-identified document type (once analysed), processing status, and an action button, giving users a batch-level view of everything they have uploaded before committing to analysis. Figure 3.3 shows this overview table populated with the three-document test batch referenced in Chapter 4, including the expandable per-document preview panel.

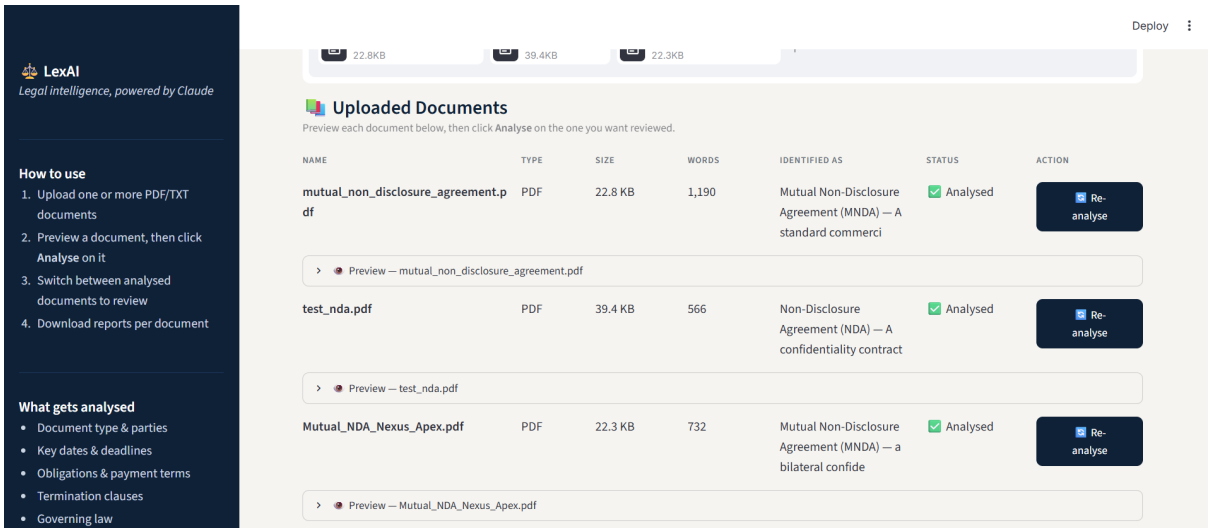


Figure 3.3: Multi-document overview table showing per-file classification, word count, and analysis status, with an expandable preview panel for each document.

3.8 PDF Text Extraction

Textual content is extracted from digitally generated PDF files using **PyPDF2**, a pure-Python library that reads each page of a PDF and retrieves its embedded text stream without depending on external binaries. PyPDF2 was selected for this stage for several concrete reasons:

- **Lightweight and dependency-free** — it introduces no system-level dependencies, which is particularly valuable inside an AWS Lambda-style constrained execution environment and keeps the Streamlit application's deployment footprint small.
- **Sufficient fidelity for born-digital documents** — for PDFs generated directly from word processors or contract-management systems (the overwhelming majority of the documents this application targets), PyPDF2 preserves reading order well enough for downstream summarization and risk analysis.
- **Simple, predictable API** — a single `PdfReader` object and a loop over `page.extract_text()` is sufficient to reconstruct the full document text, which keeps `app.py`'s extraction logic compact and easy to maintain.

The extraction function defensively falls back to an empty string for any page that yields no text (rather than raising an exception), and any document that ultimately produces no usable content — most often because it is a scanned, image-only PDF — is flagged to the user with an explicit warning rather than being silently passed on for analysis. The core extraction routine, taken from `app.py`, is shown in Listing 3.1.

```
def extract_pdf_text(file_bytes: bytes) -> str:
    reader = PyPDF2.PdfReader(io.BytesIO(file_bytes))
    return "\n".join(page.extract_text() or "" for page in reader.pages)
```

Listing 3.1: PDF text extraction using PyPDF2 (`app.py`)

Complementing this text-extraction path, `doc_preview.py` performs a separate, visual rendering of each PDF page using **PyMuPDF (fitz)**, chosen for its speed and its low-level access to a page's embedded image objects. This module inspects each page to determine whether it consists essentially of a single, full-page scanned image, and if so, calculates that image's native resolution relative to the physical page size. Pages are then rendered at whichever is higher of a comfortable baseline resolution (150 DPI) or the scan's own native resolution — capped at a safety ceiling of 300 DPI — so that scanned pages are never blurred by unnecessary upscaling, while an UnsharpMask filter from **Pillow** is selectively applied only when a low-resolution scan still had to be upscaled past its native quality. This gives users a faithful, legible in-app preview of their original document regardless of whether it was born-digital or scanned. The logic that detects a scanned page and derives its native resolution, from `doc_preview.py`, is shown in Listing 3.2.

```
def _dominant_image_native_dpi(doc, page):
    images = page.get_images(full=True)
    if not images:
```

```
        return None

    largest = None
    for img in images:
        xref = img[0]
        pix = fitz.Pixmap(doc, xref)
        area = pix.width * pix.height
        if largest is None or area > largest[0]:
            largest = (area, pix.width, pix.height)

    _, img_w, img_h = largest
    page_w_in = page.rect.width / 72.0
    page_h_in = page.rect.height / 72.0
    return min(img_w / page_w_in, img_h / page_h_in)
```

Listing 3.2: Detecting a full-page scanned image and its native DPI (`doc_preview.py`)

3.9 Text Pre-processing

Before the extracted text is handed to the AI layer, several pre-processing steps are applied to improve reliability and control cost:

- **Whitespace and encoding normalization** — extracted PDF text is stripped of leading and trailing whitespace, and plain-text uploads are decoded as UTF-8 with `errors="ignore"` so that an occasional malformed byte does not crash the entire upload.
- **Minimum-length validation** — the Lambda function rejects any document text shorter than 50 characters, preventing near-empty submissions (such as a corrupted or entirely image-based PDF) from consuming an AI API call that could never produce a meaningful result.
- **Token-budget truncation** — both the summarization and risk-analysis prompts truncate the incoming document text to its first 6,000 characters (`doc_text[:6000]`). This keeps prompt length, latency, and Claude API cost predictable and bounded, at the deliberate trade-off — discussed in Section 4.18 — of not fully analysing extremely long contracts.
- **Word-count computation** — a simple whitespace split is used to compute the document's word count, which is surfaced in the UI and embedded in the downloadable PDF report header for user context.

Listing 3.3 shows the corresponding validation and truncation logic from `lambda_function.py` and `app.py`.

```
# lambda_function.py -- minimum-length validation
doc_text = body.get("document_text", "").strip()
if not doc_text:
    return respond(400, {"error": "No document_text in request."})
if len(doc_text) < 50:
    return respond(400, {"error": "Document too short."})

word_count = len(doc_text.split())

# Prompt construction -- truncated to a fixed character budget
content=f"""...
DOCUMENT:
{doc_text[:6000]}
"""
```

Listing 3.3: Input validation and token-budget truncation

3.10 API Development

The communication contract between the Streamlit frontend and the AWS backend is implemented as a minimal, single-endpoint REST API. Amazon API Gateway was configured to accept `POST` requests carrying a JSON body of the form `{"document_text": "..."}`, and to separately respond to the `OPTIONS` pre-flight request that browsers issue automatically before a cross-origin POST — the Lambda handler explicitly checks for `event.get("httpMethod") == "OPTIONS"` and returns an empty `200 OK` response with the appropriate CORS headers, satisfying the browser's security requirements without any additional infrastructure.

Every response — success or failure — is funnelled through a single `respond()` helper function that consistently attaches the necessary CORS headers (`Access-Control-Allow-Origin`, `-Methods`, `-Headers`) and serializes the payload as JSON. Centralising response construction in this way ensures that no code path can accidentally return a malformed or CORS-incompatible response, a common and difficult-to-diagnose failure mode in Lambda-backed REST APIs. On the client side, the `call_api()` function in `app.py` issues the POST request with a generous 180-second timeout (anticipating the multi-second latency of two sequential-feeling but internally parallel LLM calls) and transparently unwraps API Gateway's response envelope, whether or not the body has been double-encoded as a JSON string.

Listing 3.4 shows the centralised response builder and CORS pre-flight handling from `lambda_function.p` and Listing 3.5 shows the corresponding client-side request function from `app.py`.

```python
def respond(status_code: int, body: dict) -> dict:
    return {
        "statusCode": status_code,
        "headers": {
            "Content-Type": "application/json",
            "Access-Control-Allow-Origin": "*",
            "Access-Control-Allow-Methods": "POST, OPTIONS",
            "Access-Control-Allow-Headers": "Content-Type",
        },
        "body": json.dumps(body),
    }

def lambda_handler(event, context):
    if event.get("httpMethod") == "OPTIONS":
        return respond(200, {"message": "OK"})
    ...
```

Listing 3.4: Centralised CORS-safe response builder and OPTIONS handling (`lambda_function.py`)

```python
def call_api(document_text: str) -> dict:
    response = requests.post(
        API_URL,
        json={"document_text": document_text},
        timeout=180,
    )
    response.raise_for_status()
    data = response.json()
    if isinstance(data.get("body"), str):
        return json.loads(data["body"])
    return data
```

Listing 3.5: Client-side API call and response-envelope unwrapping (`app.py`)

3.11 AI Model Integration (Claude)

The Claude Large Language Model is integrated through Anthropic's official `anthropic` Python SDK, instantiated once per Lambda invocation using an API key supplied through an environment variable — keeping the credential out of source control entirely. Rather than

issuing a single, broad prompt that asks the model to both summarize and assess risk simultaneously, the system deliberately splits AI analysis into two independent, purpose-built calls:

- **`parse_document()`** — issues a tightly scoped, ten-section extraction prompt (document type, parties, effective date, obligations, payment terms, confidentiality/IP, termination, dispute resolution, governing law, and special clauses) with a 2,048-token budget, instructing the model to respond in Markdown so it can be rendered natively in the Streamlit summary tab.
- **`analyse_risks()`** — issues a separate prompt with a considerably larger 8,192-token budget, since an exhaustive risk analysis of a long contract may legitimately require significantly more output than a condensed summary.

Separating these two responsibilities offers several concrete engineering advantages over a single combined prompt: each prompt can be independently tuned, versioned, and debugged; a truncation or formatting failure in one task (for example, the JSON risk array being cut off) cannot corrupt or block the other (the summary); and, critically, the two calls are dispatched *concurrently* using Python's `concurrent.futures.ThreadPoolExecutor` with two worker threads. Because both calls are I/O-bound network requests to the Claude API, running them in parallel rather than sequentially roughly halves the perceived end-to-end latency experienced by the user — a meaningful usability improvement given that legal documents can require a non-trivial amount of model "thinking time" to analyse thoroughly.

Listing 3.6 shows the concurrent dispatch logic taken from `lambda_function.py`.

```
with ThreadPoolExecutor(max_workers=2) as executor:
    future_summary = executor.submit(parse_document, doc_text)
    future_risks = executor.submit(analyse_risks, doc_text)
    summary = future_summary.result()
    risks = future_risks.result()
```

Listing 3.6: Concurrent summarization and risk-analysis calls (`lambda_function.py`)

3.12 Risk Identification Methodology

The risk-analysis prompt directs Claude to behave as an expert legal risk analyst and to return *exclusively* a JSON array — with explicit "strict rules" forbidding any surrounding prose, markdown fences, or commentary — in which every object carries exactly four keys: `clause` (a short, verbatim excerpt of the problematic text), `risk_level` (`HIGH`, `MEDIUM`, or `LOW`), `why_its_a_risk` (a plain-English rationale), and `suggestion` (a concrete protective ac-

tion). The prompt further enumerates fifteen specific categories of legal risk to guide the model's attention, including unlimited liability, one-sided indemnification, short-window auto-renewal clauses, unfavourable governing-law selection, perpetual confidentiality obligations, and unilateral modification rights — categories drawn directly from common legal due-diligence checklists rather than generic keyword flags, allowing the model to reason about *why* a clause is risky rather than merely detecting its presence. Figure 3.4 shows a rendered example of this four-field schema in the Streamlit interface, generated from an actual clause flagged in the test batch described in Chapter 4.

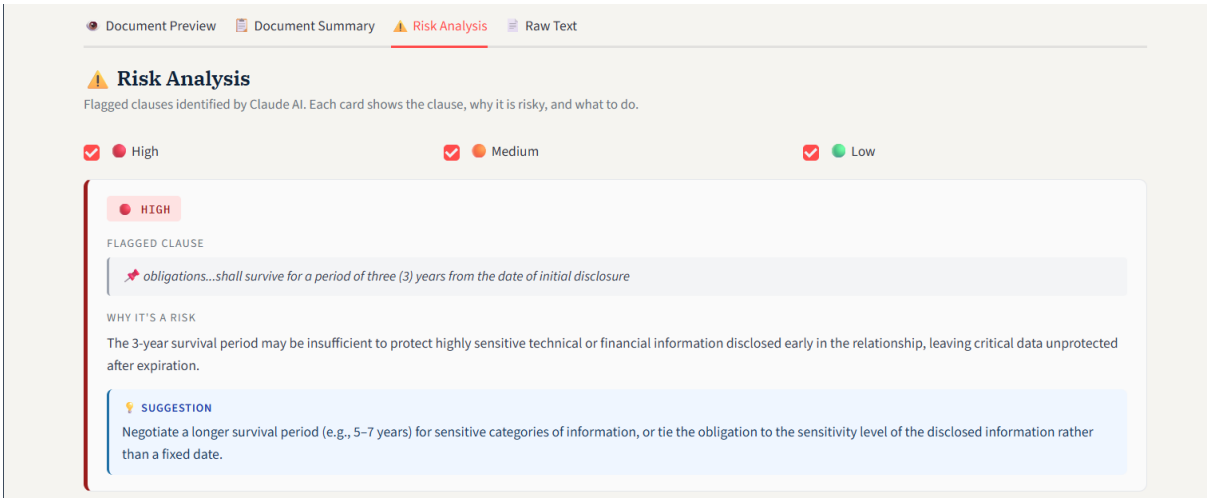


Figure 3.4: An individual High-severity risk card, showing the flagged clause excerpt, the plain-English risk rationale, and the suggested protective action.

Because Large Language Model output is not always perfectly well-formed — particularly under a large token budget where a response may be cut off mid-array — the system implements a three-tier resilience strategy rather than assuming a clean JSON parse will always succeed:

1. **Direct parse** — the raw response (after stripping any stray markdown code fences) is first parsed as JSON directly, handling both a bare array and an object wrapping the array under a `"risks"` key.

2. **Partial salvage** — if direct parsing fails, a custom bracket-depth-tracking scanner (`extract_valid_`) walks the raw text character-by-character, correctly respecting string literals and escape sequences, and reconstructs a valid JSON array out of only the *complete* risk objects present — so that if the model's response is truncated by hitting its `max_tokens` limit partway through the fifteenth risk, the first fourteen well-formed risks are still recovered and shown to the user instead of the entire analysis being discarded.

3. **Graceful degradation** — if neither strategy yields a usable result, the system returns a single explanatory placeholder risk entry (distinguishing between a truncation cause and

a genuine parsing failure) that instructs the user to consult a legal professional, ensuring the frontend never receives an empty or malformed payload it cannot render.

Every recovered risk object is subsequently passed through a `normalize_risks()` step that coerces the `risk_level` field to one of the three canonical values (defaulting unrecognised levels to `MEDIUM`) and guarantees that all four expected keys are present as strings, insulating both the Streamlit frontend and the PDF report generator from ever encountering a missing field.

Listing 3.7 shows the core of the bracket-depth-tracking salvage scanner, and Listing 3.8 shows the final schema-normalisation step, both from `lambda_function.py`.

```python
def extract_valid_risks(raw: str) -> list:
    start = raw.find("[")
    text = raw[start:]

    objs = []
    depth = 0
    obj_start = None
    in_string = False
    escape = False

    for i, ch in enumerate(text):
        if in_string:
            if escape:
                escape = False
            elif ch == "\\":
                escape = True
            elif ch == '"':
                in_string = False
            continue
        if ch == '"':
            in_string = True
        elif ch == "{":
            if depth == 1 and obj_start is None:
                obj_start = i
            depth += 1
        elif ch == "}":
            depth -= 1
            if depth == 1 and obj_start is not None:
                objs.append(text[obj_start:i + 1])
                obj_start = None
        elif ch == "[":
```

```
            depth += 1
        elif ch == "]":
            depth -= 1

    reconstructed = "[" + ",".join(objs) + "]"
    return json.loads(reconstructed)
```

Listing 3.7: Salvaging complete risk objects from a truncated JSON array (`lambda_function.py`)

```
def normalize_risks(risks: list) -> list:
    normalized = []
    for r in risks:
        if not isinstance(r, dict):
            continue
        level = str(r.get("risk_level", "MEDIUM")).strip().upper()
        if level not in ("HIGH", "MEDIUM", "LOW"):
            level = "MEDIUM"
        normalized.append({
            "clause": str(r.get("clause", "Not specified")),
            "risk_level": level,
            "why_its_a_risk": str(r.get("why_its_a_risk", "")),
            "suggestion": str(r.get("suggestion", "")),
        })
    return normalized
```

Listing 3.8: Guaranteeing a consistent risk schema before the response leaves Lambda (`lambda_function.py`)

3.13 AWS Cloud Deployment

The backend is deployed as a serverless AWS Lambda function fronted by Amazon API Gateway, a combination chosen to avoid the operational overhead of provisioning, patching, and scaling a dedicated server for a workload that is inherently bursty (document analysis requests arrive irregularly and each one is short-lived). The Lambda function's only external dependency at runtime is the `ANTHROPIC_API_KEY`, injected as an environment variable rather than hard-coded, and its error handling is deliberately mapped to specific HTTP status codes — `401` for an invalid API key, `429` when the Claude API's own rate limit is hit, `502` for upstream API or connection errors, and `500` for any unexpected failure — so that the frontend and any future API consumer can react appropriately rather than treating every failure identically.

Listing 3.9 shows how the Lambda handler maps specific Anthropic SDK exceptions to distinct HTTP status codes.

```
try:
    with ThreadPoolExecutor(max_workers=2) as executor:
        ...
except anthropic.AuthenticationError:
    return respond(401, {"error": "Invalid ANTHROPIC_API_KEY."})
except anthropic.RateLimitError:
    return respond(429, {"error": "Rate limit hit. Try again."})
except anthropic.APIStatusError as e:
    return respond(502, {"error": f"Claude API error {e.status_code}: {e.
        message}"})
except anthropic.APIConnectionError:
    return respond(502, {"error": "Could not connect to Anthropic API."})
except Exception as e:
    return respond(500, {"error": f"Unexpected error: {str(e)}"})
```

Listing 3.9: Mapping Claude API failures to specific HTTP status codes (`lambda_function.py`)

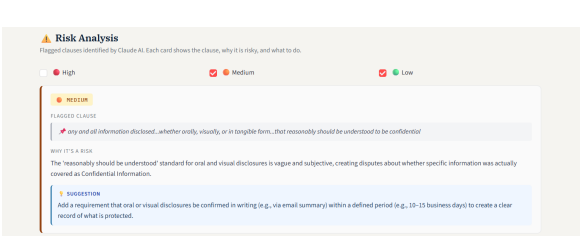
One architectural constraint identified during deployment is that Amazon API Gateway enforces a hard integration timeout of approximately 29 seconds on requests routed through it. For shorter documents this poses no issue, but as the risk-analysis token budget (8,192 tokens) is fully utilised on longer, clause-dense contracts, response generation time can approach or exceed this ceiling, risking a truncated or failed response purely due to the gateway's timeout rather than any fault in the Lambda function itself. Migrating the backend from API Gateway to a Lambda Function URL — which supports substantially longer synchronous execution windows — has therefore been identified as a planned architectural refinement to eliminate this constraint entirely, and is tracked as part of the application's ongoing AWS deployment work referenced in Section 4.18.

3.14 Streamlit User Interface

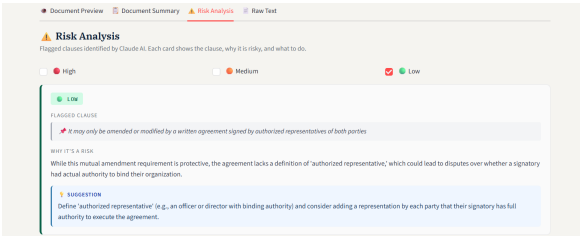
The frontend was built entirely in Streamlit, extended with a custom-injected CSS stylesheet that establishes a cohesive "LexAI" legal-technology brand identity — a navy-and-off-white palette, serif headings (IBM Plex Serif) paired with a monospace accent font (IBM Plex Mono) for data-like elements, and consistently styled metric cards, severity badges, and step indicators. Several deliberate UX design choices were implemented on top of Streamlit's base widget set:

- A persistent **sidebar** explaining how to use the application and what categories of risk are analysed, orienting first-time users without requiring separate documentation.
- A three-stage **step indicator** ("Upload → Analyse → Review") that visually communicates the user's progress through the workflow at all times.
- A **multi-document overview table** allowing users to manage an entire batch of uploads, see each file's processing status at a glance, and trigger analysis independently per document.
- A **four-tab results view** (Preview, Summary, Risk Analysis, Raw Text) that separates concerns cleanly, so a user reviewing flagged risks is not forced to scroll past the full document summary to find them.
- **Severity filter checkboxes** within the Risk Analysis tab, letting users focus exclusively on High-risk clauses when time is limited.
- A **document switcher** (radio control) that lets users move between multiple analysed documents without losing any previously generated results, since every result is retained in session state until explicitly cleared.

Figure 3.5 demonstrates the severity filter checkboxes in practice: unchecking **High** leaves only the Medium- and Low-severity cards visible (left panel), and further unchecking **Medium** narrows the view to Low-severity findings alone (right panel) — allowing a reviewer to progressively focus their attention without losing sight of how many findings exist at each level.



(a) High unchecked — Medium and Low risks shown



(b) High and Medium unchecked — Low risks only

Figure 3.5: Severity filter checkboxes in the Risk Analysis tab, letting the user progressively narrow the displayed findings by risk level.

Streamlit's reactive execution model — in which the entire script re-runs on every user interaction — was accommodated throughout by anchoring all expensive or stateful data (extracted text, AI results, OCR output) in `st.session_state`, ensuring that the rich, multi-document interface remains both responsive and consistent across reruns. Listing 3.10 shows how newly uploaded files are fingerprinted and cached, from `app.py`.

```python
for f in uploaded_files:
    key = f"{f.name}__{f.size}"
    current_keys.add(key)
    if key in st.session_state.documents:
        continue # already extracted -- don't redo work on every rerun

    file_bytes = f.read()
    if f.name.lower().endswith(".pdf"):
        document_text = extract_pdf_text(file_bytes)
    else:
        document_text = file_bytes.decode("utf-8", errors="ignore")

    st.session_state.documents[key] = {
        "name": f.name, "file_bytes": file_bytes,
        "document_text": document_text,
        "words": len(document_text.split()),
        "result": None, "risks_list": None, "error": None,
    }
```

Listing 3.10: Caching per-file state to avoid redundant re-extraction on every Streamlit rerun (`app.py`)

3.15 Result Generation

Once the Lambda function returns its response, the frontend performs a further defensive layer of result reconciliation before rendering anything to the user. The `resolve_risks()` function anticipates and gracefully handles several distinct shapes the risk data might arrive in — a clean list of dictionaries, a list whose first element unexpectedly nests another JSON array inside its `why_its_a_risk` field, a raw JSON string, or an empty result — so that a minor upstream formatting inconsistency never manifests as a crash in the user interface. Severity counts are then tallied via `count_risks_by_level()` and displayed as headline metrics, while `extract_doc_type()` uses a targeted regular expression to pull a short, human-readable document classification (e.g. "NDA", "Lease Agreement") directly out of the Claude-generated summary's first section, which is then surfaced in the document overview table for quick identification across a batch of uploads.

Listing 3.11 shows this defensive normalisation logic from `app.py`.

```python
def resolve_risks(raw_risks) -> list:
    if isinstance(raw_risks, str):
```

```
        try:
            parsed = json.loads(raw_risks.strip())
            if isinstance(parsed, list):
                return parsed
        except Exception:
            pass
        return [{"clause": "Full document", "risk_level": "MEDIUM",
                 "why_its_a_risk": raw_risks,
                 "suggestion": "Consult a legal professional."}]

    if not raw_risks:
        return []

    if isinstance(raw_risks, list) and len(raw_risks) > 0:
        first = raw_risks[0]
        if not isinstance(first, dict):
            return first if isinstance(first, list) else []

        why = first.get("why_its_a_risk", "")
        if isinstance(why, str) and why.strip().startswith("["):
            try:
                nested = json.loads(why.strip())
                if isinstance(nested, list):
                    return nested
            except Exception:
                pass
        return raw_risks

    return []
```

Listing 3.11: Defensively reconciling whatever shape the risk data arrives in (`app.py`)

The document summary itself, returned as Markdown, is passed through a light `clean_summary()` routine that strips a redundant leading `#` heading (which would otherwise render awkwardly inside the custom-styled HTML container) before being displayed inside a bordered, serif-typeset summary panel. Figure 3.6 shows the rendered Document Summary tab for the `mutual_non_discl` test document, displaying the first three of the ten structured sections generated by `parse_document()` (Section 4.11).

For a persistent, shareable output, `risk_pdf.py` renders the full risk analysis as a professionally branded PDF using **ReportLab**, chosen for its fine-grained, programmatic control over document layout — a necessity given the requirement to reproduce the same navy/severity-

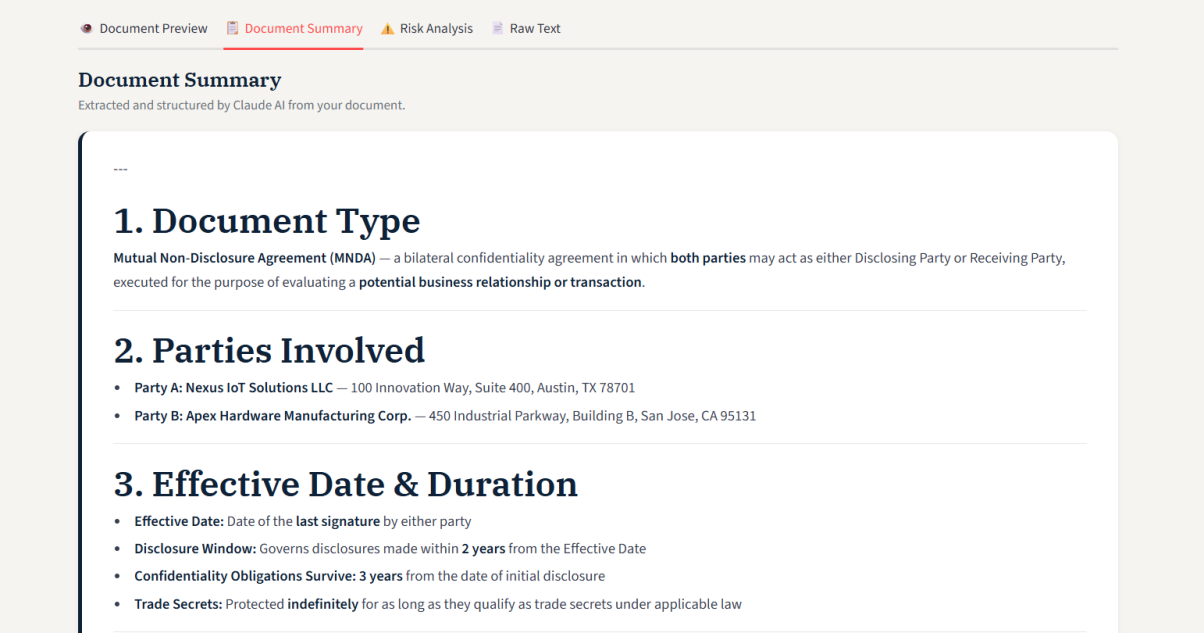


Figure 3.6: The Document Summary tab, displaying the AI-generated, section-by-section breakdown of an analysed NDA (document type, parties involved, and effective date/duration).

colour visual language used in the live Streamlit interface inside a static, downloadable artefact. The generated report includes a coloured summary table tallying High/Medium/Low risk counts, and an individually styled "risk card" for every flagged clause — each rendered as a nested ReportLab `Table`, with a coloured severity badge, the flagged clause text on a shaded background, a plain-English risk rationale, and a highlighted blue suggestion panel — mirroring, clause-for-clause, the same visual hierarchy the user has already reviewed on-screen.

3.16 Error Handling

Robust error handling was implemented at every layer of the pipeline rather than being treated as an afterthought, on the principle that a legal-document tool must fail predictably and transparently rather than silently or catastrophically:

- **Lambda / AI layer** — distinct `try/except` blocks catch `anthropic.AuthenticationError`, `anthropic.RateLimitError`, `anthropic.APIStatusError`, and `anthropic.APIConn` individually, mapping each to a specific, informative HTTP status code and message rather than a single generic "something went wrong" response; a final catch-all handles any other unexpected exception.
- **Input validation** — the Lambda handler explicitly rejects missing `document_text`, non-JSON request bodies, and documents shorter than 50 characters, returning a clear `400 Bad Request` in each case.

- **Frontend network layer** — `call_api()` invocations are wrapped to separately catch `requests.exceptions.Timeout`, `ConnectionError`, and `HTTPError`, each surfaced to the user as a distinct, human-readable message rather than a raw stack trace.
- **Document-level isolation** — because each uploaded document maintains its own `error` field in session state, a failure analysing one document (for example, a scanned PDF with no extractable text) does not affect the user's ability to view or interact with any other document in the same batch.
- **Rendering safeguards** — PDF preview rendering and PDF report generation are both wrapped in their own `try/except` blocks, so a corrupt or unusual file cannot crash the entire page — the user instead sees a contained warning message for that specific action.

Listing 3.12 shows the client-side network error handling surrounding each analysis request, from `app.py`.

```
try:
    res = call_api(d["document_text"])
    ...
except requests.exceptions.Timeout:
    st.session_state.documents[analyse_key]["error"] = "Request timed out."
except requests.exceptions.ConnectionError:
    st.session_state.documents[analyse_key]["error"] = "Could not connect to the API."
except requests.exceptions.HTTPError as e:
    st.session_state.documents[analyse_key]["error"] = f"HTTP {e.response.status_code} from API."
except Exception as e:
    st.session_state.documents[analyse_key]["error"] = str(e)
```

Listing 3.12: Frontend network error handling around the analysis request (`app.py`)

3.17 Testing and Validation

Consistent with the incremental development methodology described in Section 4.1, each module was tested independently before integration. Text extraction was validated against a range of digitally generated PDFs of varying structure (single-column contracts, multi-column agreements, documents with embedded tables) to confirm that reading order was preserved adequately for downstream AI analysis. The Lambda function's JSON-recovery logic was specifically stress-tested against artificially truncated Claude responses to confirm that the partial-

salvage algorithm described in Section 4.12 correctly recovers all complete risk objects while discarding only the final, incomplete one. The Streamlit interface was tested with simultaneous multi-document uploads to verify that session-state isolation correctly prevents cross-contamination of results between documents, and CORS configuration was validated by confirming that both the `OPTIONS` pre-flight and the subsequent `POST` request completed successfully from the deployed frontend. End-to-end testing and bug-fixing across the fully integrated, cloud-deployed system is an ongoing activity, as reflected in the current project status.

3.18 Limitations

Several limitations of the current implementation were identified during development and remain open areas for future work:

- **No automated OCR pipeline for scanned documents** — while `doc_preview.py` can detect that a page is a scanned image and allows a user to manually trigger Tesseract OCR on that single page for preview purposes, this OCR output is not yet automatically fed into the summarization and risk-analysis pipeline, meaning scanned or image-only PDFs cannot currently receive a full AI analysis.
- **No editable DOCX export with embedded AI suggestions** — reporting is currently limited to a read-only PDF risk report; generating an editable Word document with the AI's suggested clause revisions inserted directly alongside the original text remains unimplemented.
- **API Gateway timeout ceiling** — as discussed in Section 4.13, the roughly 29-second integration timeout imposed by API Gateway can constrain analysis of unusually long or clause-dense documents at the current 8,192-token risk-analysis budget.
- **Bounded document length** — both AI prompts truncate document text to its first 6,000 characters, meaning content beyond this threshold in very long contracts is not directly analysed, a deliberate cost- and latency-control trade-off rather than an oversight.
- **Session-only persistence** — all uploaded documents and analysis results are held in Streamlit's in-memory session state and are lost on a full page refresh or session expiry, since no database or persistent storage layer is currently integrated.
- **Single-provider AI dependency** — the system's summarization and risk-detection quality is entirely dependent on the availability and rate limits of the Claude API, with no fallback model configured.

- **Deployment completeness** — full production deployment of the application (as opposed to the currently functioning development-stage backend and locally run frontend) and comprehensive end-to-end testing across the complete cloud stack remain in progress.

3.19 Flowchart

Figure 3.7 illustrates the complete processing pipeline of the AI Document Parser and Risk Analyzer, from initial document upload through to final report generation.

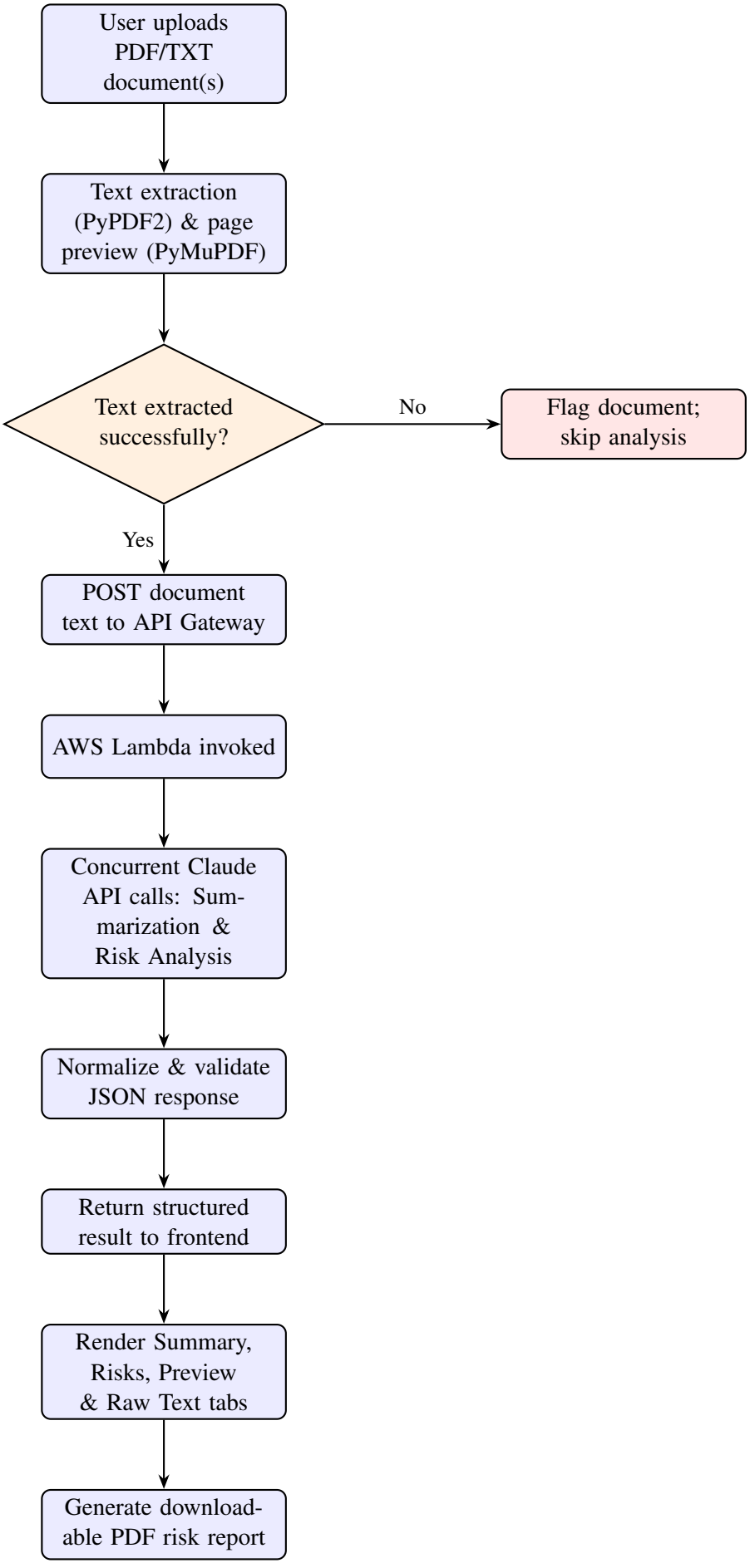


Figure 3.7: End-to-end workflow of the AI Document Parser and Risk Analyzer.

4. Data Analysis and Results

4.1 Overview of the Test Batch

To evaluate the end-to-end behaviour of the deployed pipeline, a batch of three Non-Disclosure Agreement (NDA) style documents was uploaded and analysed through the application in a single session. Table 4.1 presents the aggregate results reported by the system across the entire batch, while Table 4.2 presents a per-document breakdown, including document classification, word count, severity-wise risk counts, and total processing time.

Table 4.1: Aggregate results across the analysed document batch

Metric	**Value**
Documents Analysed	3
Total High-Severity Risks	19
Total Medium-Severity Risks	18
Total Low-Severity Risks	10
Total Risks Identified	**47**

Table 4.2: Per-document analysis results

Document	**Words**	**High**	**Medium**	**Low**	**Time (s)**
mutual_non_disclosure_agreement.pdf	1,190	4	7	3	41.0
test_nda.pdf	566	12	5	3	43.4
Mutual_NDA_Nexus_Apex.pdf	732	3	6	4	33.2
Total / Average	**829 (avg.)**	**19**	**18**	**10**	**39.2 (avg.)**

Table 4.3 shows the AI-generated document-type classification produced for each file, extracted automatically from the Claude-generated summary via `extract_doc_type()` and displayed to the user in the document-selection panel.

Table 4.3: AI-identified document classification

Document	AI-Identified Type
mutual_non_disclosure_agreement.pdf	Mutual Non-Disclosure Agreement (MNDA) – a standard commercial confidentiality agreement
test_nda.pdf	Non-Disclosure Agreement (NDA) – a confidentiality contract
Mutual_NDA_Nexus_Apex.pdf	Mutual Non-Disclosure Agreement (MNDA) – a bilateral confidentiality agreement

4.2 Descriptive Analysis

The three test documents, while all belonging broadly to the NDA family of legal instruments, varied noticeably in length and clause density – ranging from 566 words (`test_nda.pdf`) to 1,190 words (`mutual_non_disclosure_agreement.pdf`), roughly a 2.1× spread. This variation was deliberately useful for validating the pipeline, since it exercises both ends of the token-budget truncation discussed in Section 4.9 without approaching the 6,000-character ceiling for any of the three files.

Two clear observations emerge from Table 4.2. First, risk count does not scale linearly with document length: `test_nda.pdf`, the shortest document at 566 words, produced the highest number of individual risks (20) and the highest count of High-severity risks (12) of the batch, while the longest document, `mutual_non_disclosure_agreement.pdf`, produced a comparatively more moderate 14 risks with only 4 flagged as High severity. This suggests that the Claude-based risk analyser is responding to the density and one-sidedness of specific clauses (e.g. unlimited liability, unfavourable termination rights) rather than simply generating more findings as a function of input size – consistent with the semantic, context-aware analysis approach described in Section 4.12, as opposed to a naive keyword-count heuristic.

Second, processing time remained consistent and bounded across all three documents (33.2s – 43.4s, a spread of roughly 10 seconds) despite the more-than-twofold difference in document length. This is a direct, observable consequence of the concurrent summarization/risk-analysis design described in Section 4.11: because both AI calls are dispatched in parallel via `ThreadPoolExecutor` rather than sequentially, and because the risk-analysis prompt (the longer of the two AI calls) dominates total latency regardless of input length within the tested range, end-to-end response time stayed within a narrow, predictable band suitable for an interactive review workflow.

Across the batch, Medium-severity risks were the most frequently identified category in ag-

gregate (18 of 47, 38.3%), followed closely by High-severity risks (19 of 47, 40.4%), with Low-severity risks comprising the remaining 10 (21.3%). The comparatively high proportion of High-severity findings is consistent with the nature of NDAs as a document class, which frequently contain the kind of clauses the risk-analysis prompt is specifically tuned to flag as high-risk – including perpetual confidentiality obligations, broad definitions of confidential information, and asymmetric remedies – as outlined in the risk-category list in Section 4.12.

4.3 Inferential and Comparative Remarks

Given the batch size evaluated (three documents), the results presented above are best interpreted as a validation of correct end-to-end pipeline behaviour – confirming that text extraction, concurrent AI analysis, JSON normalisation, and result rendering function correctly together on real-world documents – rather than as a statistically powered study of risk-detection accuracy. No formal hypothesis testing was undertaken at this stage, and no claim of statistical significance is made regarding the differences observed between documents. A larger, more diverse test corpus spanning multiple contract categories (e.g. employment agreements, vendor contracts, lease agreements, as originally scoped in Section 4.6) would be required before drawing firm conclusions about detection accuracy, false-positive rates, or category-wise performance differences – work that is noted as a natural extension of the testing effort described in Section 4.17.

5. Conclusion

This internship set out to design and build an intelligent system capable of automating the review of legal and contractual documents – a task that is traditionally slow, labour-intensive, and highly dependent on the reviewer's own experience and attention to detail. Over the course of the internship, this objective was realised in the form of **LexAI**, a fully functional, cloud-deployed application that combines a Streamlit-based frontend, a serverless AWS Lambda backend, and the Claude Large Language Model to transform a raw legal document into a structured summary and a categorised risk report in under a minute.

Each of the project's original objectives, as set out in Chapter 2, has been substantively met. An intelligent document-parsing pipeline was built that accepts multiple PDF and TXT documents in a single batch, extracts their textual content, and renders an in-app preview without requiring the user to reopen the original file. Document summarisation was automated through a dedicated, ten-section Claude prompt that condenses lengthy agreements into a structured, readable overview. Risk analysis was implemented as a separate, concurrently executed AI task that identifies and explains legal, financial, and compliance risks in plain English, complete with concrete suggestions for mitigating each one – moving well beyond simple keyword matching to a genuinely context-aware reading of the document, as evidenced by the batch results discussed in Chapter 4. The application was deployed on a scalable, serverless cloud architecture using AWS Lambda and Amazon API Gateway, and the overall system was engineered with defensive error handling, JSON-recovery logic, and session-state management robust enough to support real, multi-document usage rather than a single-shot demonstration.

The results presented in Chapter 4, drawn from a real batch of three Non-Disclosure Agreements, offer encouraging evidence that the underlying approach works as intended: the system correctly identified 47 risks across the batch, distinguished meaningfully between High-, Medium-, and Low-severity findings, and did so within a consistent, predictable processing time regardless of document length – a direct benefit of the concurrent summarisation and risk-analysis design adopted in Section 4.11. Just as importantly, the finding that risk count did

not scale simply with document length suggests that the system is responding to the substance of specific clauses rather than merely the volume of text, which is precisely the behaviour a context-aware legal analysis tool should exhibit.

Beyond the application itself, this internship provided substantial hands-on learning across several domains that extend well beyond the classroom. Working through real integration failures – such as the JSON truncation issues addressed by the salvage-parsing logic in Section 4.12, or the CORS and API Gateway timeout considerations discussed in Sections 4.10 and 4.13 – offered a far deeper appreciation of the gap between a prototype that works once and a system engineered to fail predictably and recover gracefully. Structuring the AI prompts, dividing responsibilities between summarisation and risk analysis, and reasoning about token budgets and cost also gave direct, practical experience of prompt engineering as a discipline in its own right, distinct from general software development. Equally, the process of deploying a serverless backend on AWS, wiring it to a REST API, and connecting it to a Streamlit frontend consolidated the cloud-computing and full-stack concepts introduced during the initial training phase (Section 1.2) into a single, coherent, end-to-end system.

At the same time, this project is not presented as a finished, production-grade product, and its current limitations – detailed fully in Section 4.18 – are acknowledged candidly rather than glossed over. Scanned and image-based documents cannot yet receive a complete automated analysis, the reporting layer does not yet offer an editable, AI-annotated Word document, and full production deployment and end-to-end testing across the complete cloud stack remain ongoing work. These limitations do not detract from what has been achieved; rather, they define a clear and realistic roadmap for the system's continued development, summarised in the recommendations below.

In summary, this internship achieved its central aim: demonstrating, through a working, cloud-deployed, and empirically tested application, that the convergence of Large Language Models, serverless cloud architecture, and thoughtful software engineering can meaningfully reduce the manual effort involved in legal document review, while improving the consistency and transparency with which contractual risks are surfaced to the people who rely on them.

5.1 Recommendations for Future Work

Building on the results and limitations discussed above, the following directions are recommended for extending this project beyond the scope of the internship:

- Fully integrating Tesseract OCR into the core analysis pipeline so that scanned and

image-based legal documents can receive complete AI-based summarisation and risk analysis, rather than being limited to manual, page-by-page OCR preview.

- Extending the reporting layer to generate an editable DOCX document with the AI's suggested clause revisions inserted directly alongside the original text, complementing the existing read-only PDF risk report.
- Migrating the API layer from Amazon API Gateway to a Lambda Function URL to eliminate the roughly 29-second integration timeout constraint for larger, clause-dense documents.
- Completing full production deployment of the application on AWS and carrying out comprehensive end-to-end testing and bug-fixing across the integrated cloud stack.
- Expanding the evaluation corpus to a larger, more diverse set of contract categories – employment agreements, vendor contracts, and lease agreements – to move from pipeline validation towards a statistically meaningful assessment of risk-detection accuracy.
- Incorporating additional risk taxonomies or industry-specific compliance frameworks to broaden the system's applicability beyond general commercial contracts.
- Conducting a formal user study, ideally involving legal or paralegal professionals, to evaluate the usability, trustworthiness, and practical time-savings offered by the risk analysis output.

A. Appendices

A.1 References

The following official documentation and resources were consulted during the design, development, and testing of this project:

1. Anthropic. *Claude Developer Documentation.* https://docs.claude.com
2. Anthropic. *Claude API Reference – Messages.* https://docs.claude.com/en/api/messages
3. Amazon Web Services. *AWS Lambda Developer Guide.* https://docs.aws.amazon.com/lambda/
4. Amazon Web Services. *Amazon API Gateway Developer Guide.* https://docs.aws.amazon.com/apigateway/
5. Streamlit Inc. *Streamlit Documentation.* https://docs.streamlit.io
6. PyPDF2 Contributors. *PyPDF2 Documentation.* https://pypdf2.readthedocs.io
7. Artifex Software / PyMuPDF Contributors. *PyMuPDF (fitz) Documentation.* https://pymupdf.readthedocs.io
8. ReportLab. *ReportLab PDF Library User Guide.* https://www.reportlab.com/docs/reportlab-userguide.pdf
9. Python Software Foundation. *concurrent.futures – Launching Parallel Tasks.* https://docs.python.org/3/library/concurrent.futures.html
10. Tesseract OCR Contributors. *Tesseract Open Source OCR Engine.* https://github.com/tesseract-ocr/tesseract
11. Amazon Web Services. *Cross-Origin Resource Sharing (CORS) for REST APIs in API*

Gateway. https://docs.aws.amazon.com/apigateway/latest/developerguide/how-to-cors.html

A.2 GitHub Repository

The complete source code developed during this internship, including the Streamlit frontend, AWS Lambda backend, and supporting modules, is publicly available at:

https://github.com/Soureen25/LexAI/tree/main

A.3 Supplementary Materials

The following supplementary materials accompany this report:

- **Digital copy of this report:** https://drive.google.com/file/d/1cnit9e2w76T3_QrLz8YXNS2i7cQgwqw4/view?usp=sharing
- **Sample test documents:** the three NDA-style documents used for the evaluation in Chapter 4 (`mutual_non_disclosure_agreement.pdf`, `test_nda.pdf`, `Mutual_NDA_Ne` – https://github.com/Soureen25/LexAI/tree/main/sample_documents
- **Demonstration video:** a short screen recording of the upload → analyse → review workflow – https://drive.google.com/file/d/1ZWdF9gnsFAPjG6SdPd0ilw3hvjTMdquN view?usp=sharing
- **Full-resolution architecture diagram:** a higher-resolution version of Figure 3.1 – https://github.com/Soureen25/LexAI/blob/main/screenshots/architechture_flow/LexAI_Architecture_Diagram.png